

Python/NumPy/Matplotlib to MATLAB Quick Reference

A compact syntax guide for students moving from Python to MATLAB

Main idea: NumPy arrays and MATLAB matrices are very similar, but indexing differs: Python starts at **0**, MATLAB starts at **1**. Python uses functions such as `np.sin(x)`, whereas MATLAB often uses nearly the same mathematical name, e.g. `sin(x)`.

1. Variables, vectors, and matrices

Operation	Python / NumPy	MATLAB
Scalar	<code>x = 3.0</code>	<code>x = 3.0;</code>
Row vector	<code>x = np.array([1,2,3])</code>	<code>x = [1 2 3];</code>
Column vector	<code>x = np.array([[1],[2],[3]])</code>	<code>x = [1; 2; 3];</code>
Matrix	<code>A = np.array([[1,2],[3,4]])</code>	<code>A = [1 2; 3 4];</code>
Zeros	<code>np.zeros((3,4))</code>	<code>zeros(3,4)</code>
Ones	<code>np.ones((3,4))</code>	<code>ones(3,4)</code>
Identity matrix	<code>np.eye(4)</code>	<code>eye(4)</code>
Sequence with step	<code>np.arange(0,10,2)</code>	<code>0:2:8</code>
Equally spaced values	<code>np.linspace(0,1,6)</code>	<code>linspace(0,1,6)</code>
Matrix size	<code>A.shape</code>	<code>size(A)</code>
Number of elements	<code>A.size</code>	<code>numel(A)</code>
Transpose	<code>A.T</code>	<code>A.'</code>

2. Indexing and slicing

Operation	Python / NumPy	MATLAB
First element	<code>x[0]</code>	<code>x(1)</code>
Third element	<code>x[2]</code>	<code>x(3)</code>
First row	<code>A[0,:]</code>	<code>A(1,:)</code>
Second column	<code>A[:,1]</code>	<code>A(:,2)</code>
Rows 2–4	<code>A[1:4,:]</code>	<code>A(2:4,:)</code>
Last element	<code>x[-1]</code>	<code>x(end)</code>
Every second element	<code>x[::2]</code>	<code>x(1:2:end)</code>
Elements larger than 0	<code>x[x > 0]</code>	<code>x(x > 0)</code>

Important: Python slices exclude the final index. For example, `x[1:4]` contains indices 1, 2, and 3. MATLAB ranges include the final index, so `x(2:4)` contains elements 2, 3, and 4.

3. Basic mathematical operations

Operation	Python / NumPy	MATLAB
Addition	<code>A + B</code>	<code>A + B</code>
Element-by-element multiply	<code>A * B</code>	<code>A .* B</code>
Matrix multiplication	<code>A @ B</code>	<code>A * B</code>
Element-by-element division	<code>A / B</code>	<code>A ./ B</code>
Element-by-element power	<code>A**2</code>	<code>A.^2</code>
Square root	<code>np.sqrt(x)</code>	<code>sqrt(x)</code>
Exponential	<code>np.exp(x)</code>	<code>exp(x)</code>
Sine / cosine	<code>np.sin(x), np.cos(x)</code>	<code>sin(x), cos(x)</code>
Absolute value	<code>np.abs(x)</code>	<code>abs(x)</code>
Sum	<code>np.sum(x)</code>	<code>sum(x)</code>
Mean	<code>np.mean(x)</code>	<code>mean(x)</code>
Maximum	<code>np.max(x)</code>	<code>max(x)</code>
Matrix inverse	<code>np.linalg.inv(A)</code>	<code>inv(A)</code>
Solve $Ax = b$	<code>x = np.linalg.solve(A,b)</code>	<code>x = A\b;</code>

4. Functions, loops, and conditions

Operation	Python	MATLAB
Simple function return x**2 + 1	def f(x): function y = f(x) y = x.^2 + 1;	end
For loop print(i)	for i in range(5): for i = 1:5 disp(i)	end
If statement y = 1 else: y = -1	if x > 0: if x > 0 y = 1; else y = -1; end	

5. Plotting: Matplotlib and MATLAB

Operation	Python / Matplotlib	MATLAB
Basic line plot	plt.plot(x,y); plt.show()	plot(x,y)
Line with symbols	plt.plot(x,y,'o-')	plot(x,y,'o-')
Axis labels	plt.xlabel('Time'); plt.ylabel('Amplitude')	xlabel('Time'); ylabel('Amplitude')
Title	plt.title('Signal')	title('Signal')
Grid	plt.grid(True)	grid on
Axis limits	plt.xlim(0,10)	xlim([0 10])
Two curves	plt.plot(x,y1); plt.plot(x,y2)	plot(x,y1); hold on; plot(x,y2)
Legend	plt.legend(['A', 'B'])	legend('A', 'B')
Image / matrix	plt.imshow(A, aspect='auto')	imagesc(A)
Colorbar	plt.colorbar()	colorbar

6. A small side-by-side example

Python / NumPy / Matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 200)
y = np.sin(x)

A = np.array([[1,2],[3,4]])
b = np.array([1,2])
u = np.linalg.solve(A,b)

plt.plot(x,y)
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.grid(True)
plt.show()
```

MATLAB

```
x = linspace(0,2*pi,200);
y = sin(x);

A = [1 2; 3 4];
b = [1; 2];
u = A\b;

plot(x,y)
xlabel('x')
ylabel('sin(x)')
grid on
```

MATLAB reminders:

- A semicolon suppresses output: `x = 3;` versus `x = 3`.
- Use `.*`, `./`, and `.^` for element-by-element operations.
- Use `*` for matrix multiplication.
- MATLAB indexing begins at 1; Python indexing begins at 0.